DSAI541 Assignment 02

March 7, 2026

0.1 Assignment 02 - Deadline: Mar 11, 2026, Wed 11pm

DSAI 541 Spring 2026 Complete the assignment below and upload both the .ipynb file and its pdf to https://moodle.bogazici.edu.tr by the deadline given above. The submission page on Moodle will close automatically after this date and time and no late homeworks will be accepted.

To make a pdf, this may work if you're using Jupyter notebook on browser: Hit CMD+P or CTRL+P, and save it as PDF. You may also use other options from the File menu. (You can use Google Colab, but at the end you should download the ipynb notebook to submit it on Moodle.)

[24]:
```
# Run this cell first

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from IPython.display import HTML

# Set the display option to show all rows scrolling with a slider
pd.set_option('display.max_rows', None)
# To disable this, run the line below:
# pd.reset_option('display.max_rows')
```

1 Problem 1: Trying different optimizers (100 pts)

You have to use PyTorch in this problem, Tensorflow will not be accepted as answer.

In this homework, you will repeat the PyTorch training you did in Assignment 01 with different optimizers.

Set `epoch = 100`. You can choose other hyperparameters as you like. The ANN architecture to be used is given below.

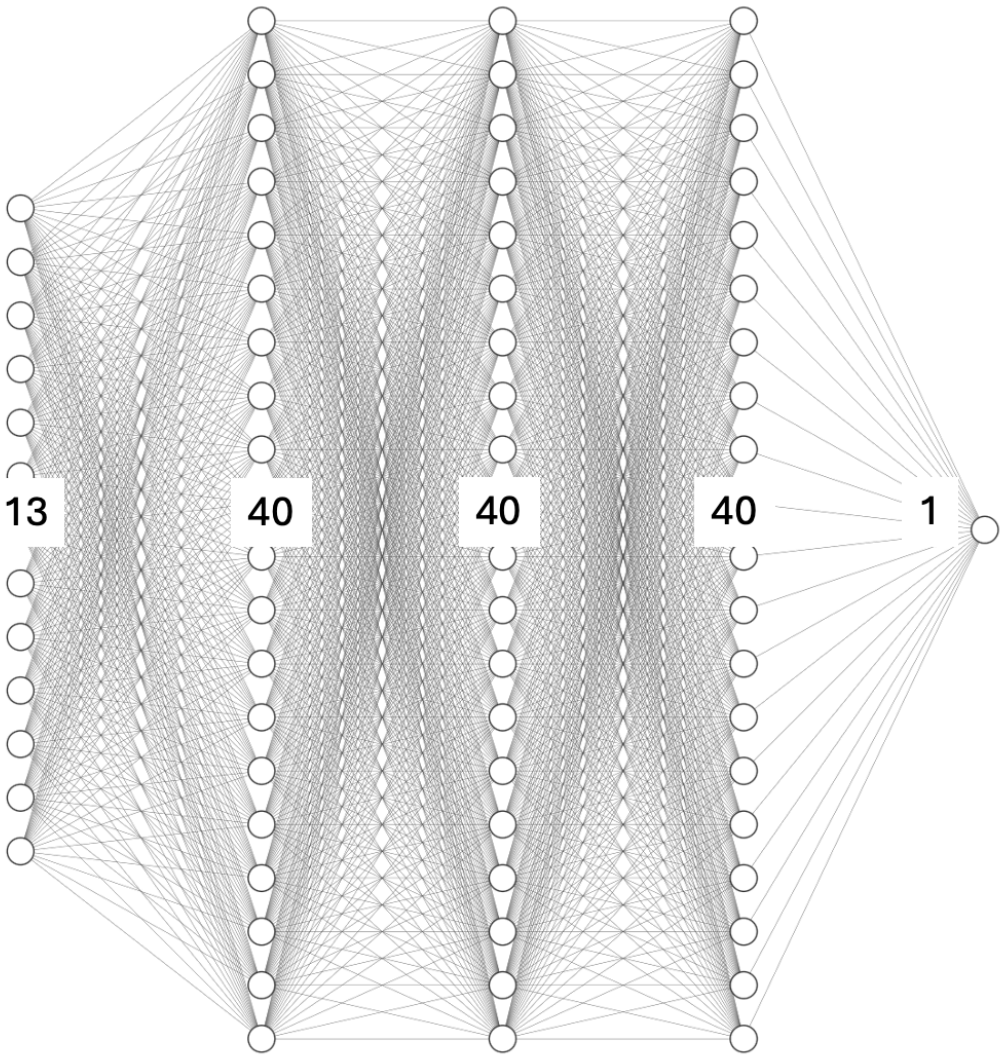
a) Train the ANN model with PyTorch as in Assignment 1 using RMSprop optimizer, where you used `Lecture 02.ipynb`.

b) Train the ANN model with PyTorch as in Assignment 1 but use Stochastic Gradient Descent [optim.SGD()] with no momentum.

c) Train the ANN model with PyTorch as in Assignment 1 but use Stochastic Gradient Descent [optim.SGD()] with momentum.

d) Train the ANN model with PyTorch as in Assignment 1 but use ADAM optimizer.

e) Discuss which training approach led to minimum train loss at the end of 100 epochs.



This part is directly coppied from assignment #1

[25]:
```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("simpleparadox/bostonhousingdataset")

print("Path to dataset files:", path)
```

```
Path to dataset files: C:\Users\baldi\.cache\kagglehub\datasets\simpleparadox\bo
stonhousingdataset\versions\1
```

[26]:
```
import os
# Load the dataset into a pandas DataFrame
```

```
csv_file = os.path.join(path, "boston-housing-dataset.csv")
dataset = pd.read_csv(csv_file)
dataset.head()
```

[26]:

```
   Unnamed: 0     CRIM    ZN  INDUS  CHAS    NOX     RM   AGE     DIS  RAD  \
0           0  0.00632  18.0   2.31     0  0.538  6.575  65.2  4.0900    1
1           1  0.02731   0.0   7.07     0  0.469  6.421  78.9  4.9671    2
2           2  0.02729   0.0   7.07     0  0.469  7.185  61.1  4.9671    2
3           3  0.03237   0.0   2.18     0  0.458  6.998  45.8  6.0622    3
4           4  0.06905   0.0   2.18     0  0.458  7.147  54.2  6.0622    3

     TAX  PTRATIO       B  LSTAT  MEDV
0  296.0     15.3  396.90   4.98  24.0
1  242.0     17.8  396.90   9.14  21.6
2  242.0     17.8  392.83   4.03  34.7
3  222.0     18.7  394.63   2.94  33.4
4  222.0     18.7  396.90   5.33  36.2
```

[27]:

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import TensorDataset
from torch.utils.data import DataLoader, random_split
from sklearn.preprocessing import StandardScaler

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

```
Using device: cuda
```

[28]:

```
X = dataset.iloc[:, 1:-1].values # Extract features directly from the␣
↪DataFrame, excluding the first column (ID) and the last column (target)
y = dataset.iloc[:, -1].values  # Extract target labels

full_size = len(dataset)
train_size = int(0.65 * full_size)
val_size = int(0.15 * full_size)

# Shuffle and split the indices FIRST
indices = np.random.permutation(full_size)
train_idx = indices[:train_size]
val_idx = indices[train_size:train_size+val_size]
test_idx = indices[train_size+val_size:]

# Now we are ready to scale the features using StandardScaler, fitting only on␣
↪the training data
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X[train_idx])
X_val = scaler.transform(X[val_idx])
X_test = scaler.transform(X[test_idx])

# we can now create TensorDatasets for training, validation, and testing using␣
↪the scaled features and corresponding labels
train_dataset = TensorDataset(torch.tensor(X_train, dtype=torch.float32), torch.
↪tensor(y[train_idx], dtype=torch.float32).view(-1, 1))
val_dataset = TensorDataset(torch.tensor(X_val, dtype=torch.float32), torch.
↪tensor(y[val_idx], dtype=torch.float32).view(-1, 1))
test_dataset = TensorDataset(torch.tensor(X_test, dtype=torch.float32), torch.
↪tensor(y[test_idx], dtype=torch.float32).view(-1, 1))
```

[29]:
```
# And also, we are creating the DataLoaders for training, validation.
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=32)
test_loader = DataLoader(test_dataset, batch_size=32)
```

[30]:
```
# To be sure about the shapes of the batches, we can print the shape of the␣
↪first batch from the train_loader
print(next(iter(train_loader))[0].shape)  # X batch
```

```
torch.Size([32, 13])
```

[31]:
```
class BostonHousingNN(nn.Module):
    """
    Neural network architecture for the dataset. As wanted in the assignment,
    it consists of 3 hidden layers with 40 neurons each and ReLU activation,
    and an output layer with a single neuron for regression.
    """
    def __init__(self):
        super(BostonHousingNN, self).__init__()
        self.fc1 = nn.Linear(13, 40) # 13->40 | 32x13 -> 32x40
        self.fc2 = nn.Linear(40, 40) # 40->40 | 32x40 -> 32x40
        self.fc3 = nn.Linear(40, 40) # 40->40 | 32x40 -> 32x40
        self.fc4 = nn.Linear(40, 1)  # 40->1  | 32x40 -> 32x1

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = F.relu(self.fc3(x))
        x = self.fc4(x)
        return x


def train_model(model, train_loader, val_loader, criterion, optimizer,␣
↪scheduler=None, num_epochs=100):
```

```
    train_losses = []
    val_losses = []
    last_10_epochs_train_losses = []
    for epoch in range(num_epochs):
        model.train() # We are setting the model into training mode at the␣
↪beginning of each epoch
        epoch_train_loss = 0
        epoch_val_loss = 0
        for batch in train_loader:
            X_batch, y_batch = batch
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
            optimizer.zero_grad() # We don't want gradients to accumulate␣
↪across batches, so we zero them at the start of each batch
            outputs = model(X_batch)
            loss = criterion(outputs, y_batch)
            loss.backward() # BackProp
            optimizer.step() # Update weights
            epoch_train_loss += loss.item()
        train_losses.append(epoch_train_loss / len(train_loader))
        if scheduler:
            scheduler.step()
        if epoch % 20 == 0: # Print loss every 20 epochs
            print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {epoch_train_loss/
↪len(train_loader):.4f}')
        model.eval() # We are setting the model into evaluation mode before␣
↪running validation, to disable dropout and batch norm updates
        with torch.no_grad():
            for batch in val_loader:
                X_batch, y_batch = batch
                X_batch, y_batch = X_batch.to(device), y_batch.to(device)
                outputs = model(X_batch)
                val_loss = criterion(outputs, y_batch)
                epoch_val_loss += val_loss.item()
        val_losses.append(epoch_val_loss / len(val_loader))
        if epoch % 20 == 0: # Print loss every 20 epochs
            print(f'Epoch [{epoch+1}/{num_epochs}], Val Loss: {epoch_val_loss/
↪len(val_loader):.4f}')

    return train_losses, val_losses
```

Part a)

[32]:
```
model = BostonHousingNN().to(device)
criterion = nn.MSELoss()
optimizer = torch.optim.RMSprop(model.parameters(), lr=0.001)
```

[33]:
```
import time
start_time = time.perf_counter()
train_losses_rms_prop, val_losses_rms_prop = train_model(model, train_loader,␣
↪val_loader, criterion, optimizer, num_epochs=100)
end_time = time.perf_counter()
print(f"Training completed in {end_time - start_time:.2f} seconds")
```

```
Epoch [1/100], Loss: 536.7426
Epoch [1/100], Val Loss: 268.0768
Epoch [21/100], Loss: 12.3299
Epoch [21/100], Val Loss: 20.7534
Epoch [41/100], Loss: 9.4784
Epoch [41/100], Val Loss: 23.8157
Epoch [61/100], Loss: 9.3487
Epoch [61/100], Val Loss: 38.6029
Epoch [81/100], Loss: 6.7573
Epoch [81/100], Val Loss: 22.9814
Training completed in 2.30 seconds
```

Part b)

[34]:
```
model = BostonHousingNN().to(device)
criterion = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.001)
```

[35]:
```
import time
start_time = time.perf_counter()
train_losses_SGD_wo_mom, val_losses_SGD_wo_mom = train_model(model,␣
↪train_loader, val_loader, criterion, optimizer, num_epochs=100)
end_time = time.perf_counter()
print(f"Training completed in {end_time - start_time:.2f} seconds")
```

```
Epoch [1/100], Loss: 578.6192
Epoch [1/100], Val Loss: 530.6817
Epoch [21/100], Loss: 9.9471
Epoch [21/100], Val Loss: 17.8725
Epoch [41/100], Loss: 8.6269
Epoch [41/100], Val Loss: 19.8458
Epoch [61/100], Loss: 7.0791
Epoch [61/100], Val Loss: 19.5621
Epoch [81/100], Loss: 6.7862
Epoch [81/100], Val Loss: 25.5567
Training completed in 2.03 seconds
```

Part c)

[36]:
```
model = BostonHousingNN().to(device)
criterion = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
```

[37]:
```
import time
start_time = time.perf_counter()
train_losses_SGD_with_mom, val_losses_SGD_with_mom = train_model(model,␣
↪train_loader, val_loader, criterion, optimizer, num_epochs=100)
end_time = time.perf_counter()
print(f"Training completed in {end_time - start_time:.2f} seconds")
```

```
Epoch [1/100], Loss: 572.6635
Epoch [1/100], Val Loss: 411.9079
Epoch [21/100], Loss: 7.2071
Epoch [21/100], Val Loss: 30.7633
Epoch [41/100], Loss: 7.5449
Epoch [41/100], Val Loss: 14.7266
Epoch [61/100], Loss: 5.1298
Epoch [61/100], Val Loss: 20.7897
Epoch [81/100], Loss: 3.0912
Epoch [81/100], Val Loss: 23.3665
Training completed in 2.16 seconds
```

[38]:
```
model = BostonHousingNN().to(device)
criterion = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.001, momentum=0.9,␣
↪nesterov=True)
```

[39]:
```
import time
start_time = time.perf_counter()
train_losses_SGD_with_mom_and_nesterov, val_losses_SGD_with_mom_and_nesterov, =␣
↪train_model(model, train_loader, val_loader, criterion, optimizer,␣
↪num_epochs=100)
end_time = time.perf_counter()
print(f"Training completed in {end_time - start_time:.2f} seconds")
```

```
Epoch [1/100], Loss: 531.2956
Epoch [1/100], Val Loss: 198.2891
Epoch [21/100], Loss: 7.4264
Epoch [21/100], Val Loss: 29.9740
Epoch [41/100], Loss: 3.9632
Epoch [41/100], Val Loss: 22.3671
Epoch [61/100], Loss: 3.1261
Epoch [61/100], Val Loss: 22.5620
Epoch [81/100], Loss: 2.7142
Epoch [81/100], Val Loss: 19.6968
Training completed in 2.14 seconds
```

Part d)

[40]:
```
model = BostonHousingNN().to(device)
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

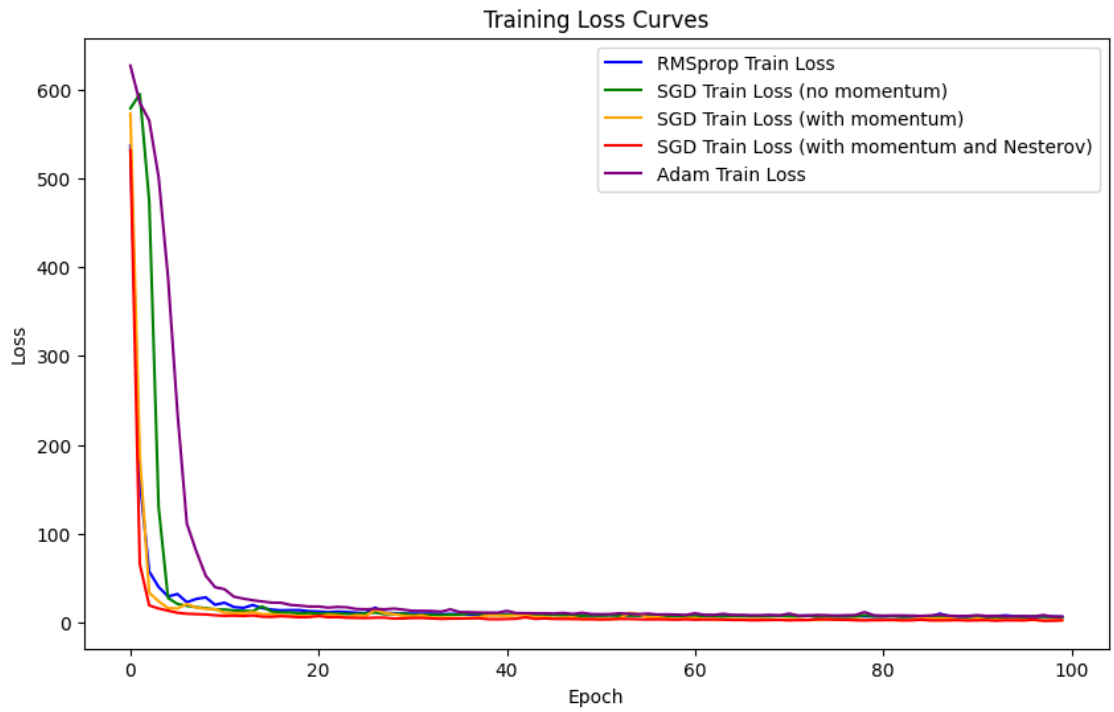
[41]:
```
import time
start_time = time.perf_counter()
train_losses_ADAM, val_losses_ADAM = train_model(model, train_loader,␣
↪val_loader, criterion, optimizer, num_epochs=100)
end_time = time.perf_counter()
print(f"Training completed in {end_time - start_time:.2f} seconds")
```

```
Epoch [1/100], Loss: 626.6004
Epoch [1/100], Val Loss: 546.0285
Epoch [21/100], Loss: 18.0412
Epoch [21/100], Val Loss: 18.4201
Epoch [41/100], Loss: 13.0687
Epoch [41/100], Val Loss: 21.8504
Epoch [61/100], Loss: 10.4109
Epoch [61/100], Val Loss: 25.9063
Epoch [81/100], Loss: 7.6432
Epoch [81/100], Val Loss: 26.0665
Training completed in 2.46 seconds
```

Part e)

- As we can see in the figure, the training corresponding to the red line (SGD with momentum and Nesterov) is the fastest. Nesterov update is another binary hyperparameter we need to determine before setting the SGD optimizer with momentum. It chooses the point we calculate the gradient.
- Second fastest convergent optimizer is again SGD with momentum but this time without nesterov update.
- Third one is RMSprop this time, however, because of the sthocastic nature of the SGD, if model is trained with some good batch, SGD can beat RMSprop in terms of convergence speed, as I keep experimenting the behaviours, I kept facing with that.
- The Adam is the one which converges slowest all the time.

[42]:
```
fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(train_losses_rms_prop, label='RMSprop Train Loss', color='blue')
ax.plot(train_losses_SGD_wo_mom, label='SGD Train Loss (no momentum)',␣
↪color='green')
ax.plot(train_losses_SGD_with_mom, label='SGD Train Loss (with momentum)',␣
↪color='orange')
ax.plot(train_losses_SGD_with_mom_and_nesterov, label='SGD Train Loss (with␣
↪momentum and Nesterov)', color='red')
ax.plot(train_losses_ADAM, label='Adam Train Loss', color='purple')
ax.set_title('Training Loss Curves')
ax.set_xlabel('Epoch')
ax.set_ylabel('Loss')
ax.legend()
plt.show()
```



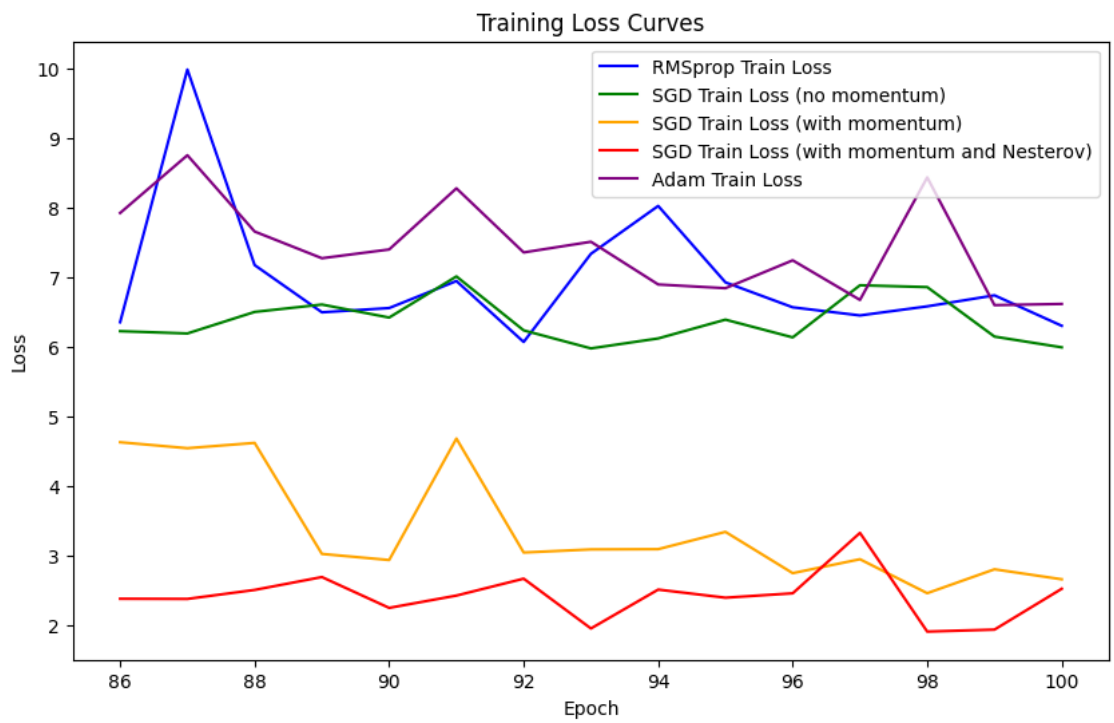
To answer the question which optimizer led to the lowest training loss, let's zoom in to the tail of the loss curves.

- We see that SGD with momentum and with nesterov are the best two optimizers among the others in this setting.
- SGD w/o momentum is slightly better than remaining two of the optimizers. It generates a stabil curve around loss value 6.
- Adam and RMSprop performs equally well, they alternate and worst performing two optimizers.

To obtain better results and be more certain about the findings, we should repeate this experiment over and over again, and then conclude our hypothesis.

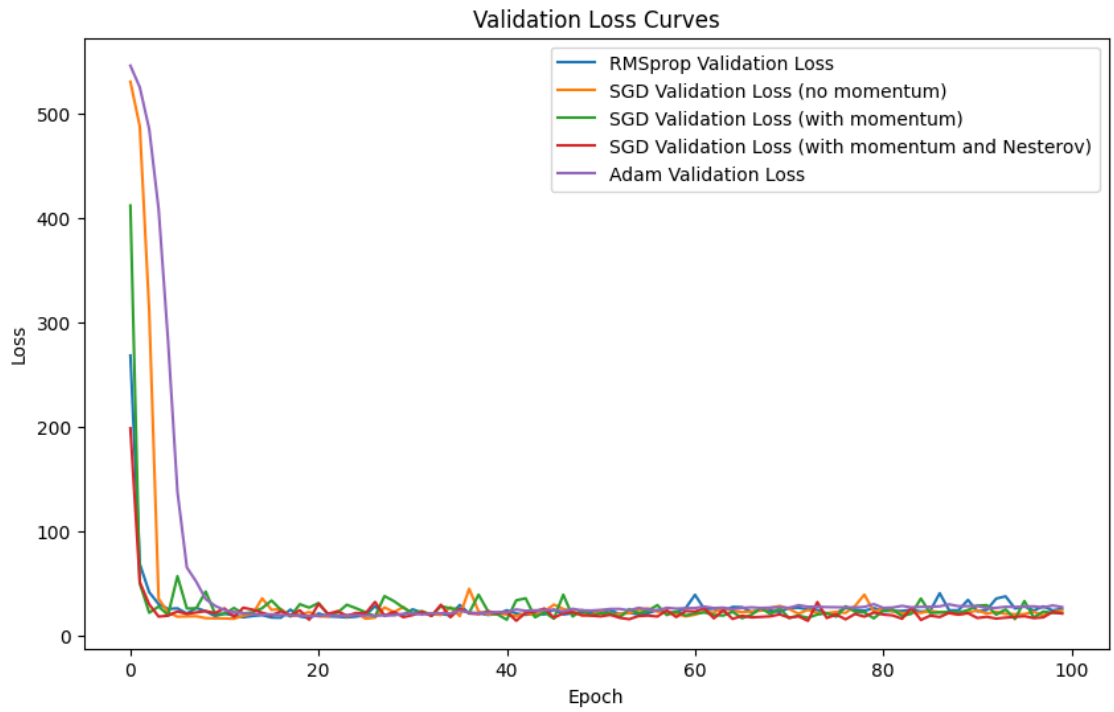
[48]:
```
fig, ax = plt.subplots(figsize=(10, 6))
epochs = range(86, 86 + 15)
ax.plot(epochs, train_losses_rms_prop[-15:], label='RMSprop Train Loss',␣
 ↪color='blue')
ax.plot(epochs, train_losses_SGD_wo_mom[-15:], label='SGD Train Loss (no␣
 ↪momentum)', color='green')
ax.plot(epochs, train_losses_SGD_with_mom[-15:], label='SGD Train Loss (with␣
 ↪momentum)', color='orange')
ax.plot(epochs, train_losses_SGD_with_mom_and_nesterov[-15:], label='SGD Train␣
 ↪Loss (with momentum and Nesterov)', color='red')
```

```
ax.plot(epochs, train_losses_ADAM[-15:], label='Adam Train Loss',␣
 ↪color='purple')
ax.set_title('Training Loss Curves')
ax.set_xlabel('Epoch')
ax.set_ylabel('Loss')
ax.legend()
plt.show()
```



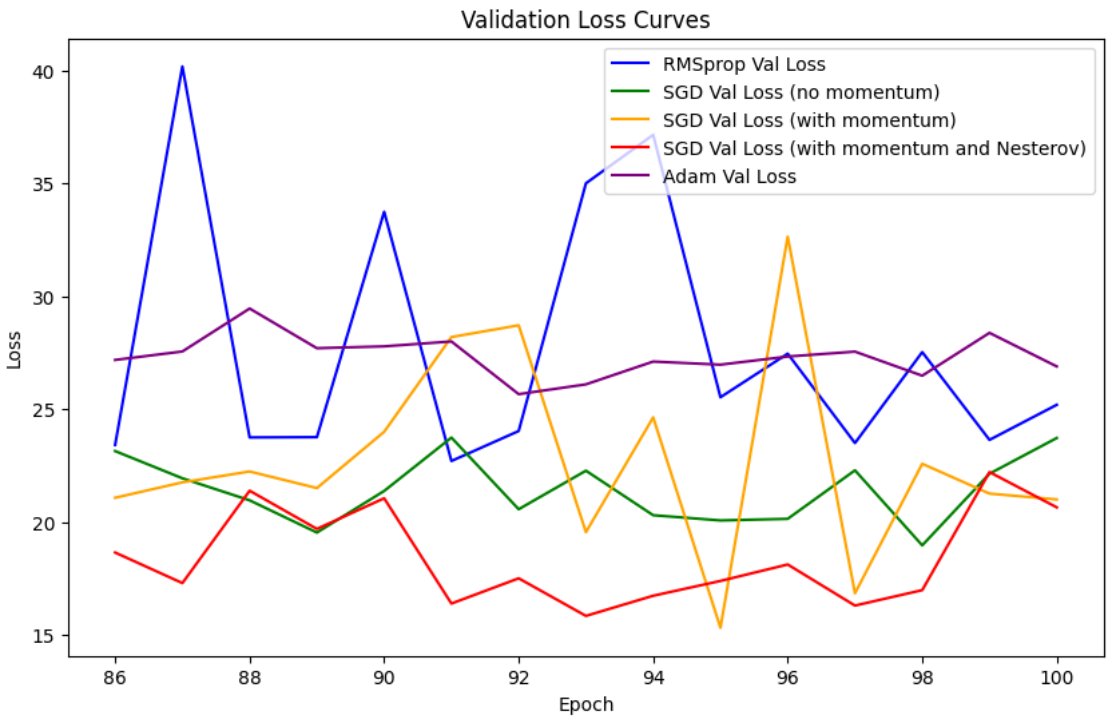
Similar loss curve appears when we investigate the validation losses.

[45]:
```
fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(val_losses_rms_prop, label='RMSprop Validation Loss')
ax.plot(val_losses_SGD_wo_mom, label='SGD Validation Loss (no momentum)')
ax.plot(val_losses_SGD_with_mom, label='SGD Validation Loss (with momentum)')
ax.plot(val_losses_SGD_with_mom_and_nesterov, label='SGD Validation Loss (with␣
 ↪momentum and Nesterov)')
ax.plot(val_losses_ADAM, label='Adam Validation Loss')
ax.set_title('Validation Loss Curves')
ax.set_xlabel('Epoch')
ax.set_ylabel('Loss')
ax.legend()
plt.show()
```



The validation losses confirms our previous findings, SGD with nesterov is the best among others. The only interesting and different case when we compare this results with the results obtained by comparing training loss curves is SGD w/o momentum generates more stabil and validation loss then SGD with momentum, also average loss of SGD w/o momentum is lower than the version with momentum.

[47]:
```
fig, ax = plt.subplots(figsize=(10, 6))
epochs = range(86, 86 + 15)
ax.plot(epochs, val_losses_rms_prop[-15:], label='RMSprop Val Loss',␣
 ↪color='blue')
ax.plot(epochs, val_losses_SGD_wo_mom[-15:], label='SGD Val Loss (no␣
 ↪momentum)', color='green')
ax.plot(epochs, val_losses_SGD_with_mom[-15:], label='SGD Val Loss (with␣
 ↪momentum)', color='orange')
ax.plot(epochs, val_losses_SGD_with_mom_and_nesterov[-15:], label='SGD Val Loss␣
 ↪(with momentum and Nesterov)', color='red')
ax.plot(epochs, val_losses_ADAM[-15:], label='Adam Val Loss', color='purple')
ax.set_title('Validation Loss Curves')
ax.set_xlabel('Epoch')
ax.set_ylabel('Loss')
ax.legend()
plt.show()
```


Validation Loss Curves
RMSprop Val Loss
SGD Val Loss (no momentum)
SGD Val Loss (with momentum)
SGD Val Loss (with momentum and Nesterov)
Adam Val Loss
Loss
Epoch
40
35
30
25
20
15
86
88
90
92
94
96
98
100